INTERVIEW PREP SERIES

# C++

## Interview Questions & Easy Answers

---

**52 questions · 28 marked ★ most asked**

Covers C++ basics, pointers and references, memory (new/delete, malloc/calloc), OOP pillars, constructors and destructors, virtual functions and polymorphism, friend and static, templates, STL containers and algorithms, smart pointers, exceptions and modern C++ (11/14/17/20). Includes the OOP questions reported in TCS, UKG and embedUR technical rounds.

### How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

# Basics

### Q1. What is C++? How is it different from C? ★ Most asked

C++ is a general-purpose, compiled language created by Bjarne Stroustrup as an extension of C. Key differences:

- C is procedural; C++ supports procedural **and** object-oriented programming (classes, inheritance, polymorphism).
- C++ adds references, function and operator overloading, templates, exceptions, namespaces and the STL.
- C uses `malloc/free` and `printf/scanf`; C++ adds `new/delete` and `cin/cout`.

### Q2. What is the difference between C++ and Java? ★ Most asked

- C++ compiles to machine code (platform-dependent); Java compiles to bytecode that runs on the JVM (platform-independent).
- C++ has pointers and manual memory management; Java has no explicit pointers and uses garbage collection.
- C++ supports multiple inheritance of classes; Java only via interfaces.
- C++ supports operator overloading; Java doesn't.
- C++ is usually faster; Java is easier to make safe and portable.

### Q3. What are the basic data types in C++?

`int`, `float`, `double`, `char`, `bool`, `void`, plus modifiers `short`, `long`, `signed`, `unsigned`. Typical sizes: `char` 1 byte, `int` 4 bytes, `double` 8 bytes (they can vary by system; check with `sizeof`).

### Q4. What does `static` mean in C++? ★ Most asked

It depends on where it's used:

- **Static local variable**: keeps its value between function calls; created once.
- **Static global variable/function**: visible only inside that file.
- **Static data member**: shared by all objects of the class (one copy).
- **Static member function**: belongs to the class, not objects; can only access static members; called as `ClassName::func()`.

```
void counter() {
  static int count = 0;   // initialised only once
  count++;
  cout << count;          // 1, 2, 3... on each call
}
```

### Q5. What is the difference between `const` and `#define`?

`#define PI 3.14` is a preprocessor text replacement with no type and no scope. `const double PI = 3.14;` has a type, follows scope rules and is checked by the compiler. Prefer `const` or `constexpr`.

### Q6. What is a namespace?

A namespace groups names (variables, functions, classes) to avoid name clashes. The standard library lives in `std`, so you write `std::cout`. `using namespace std;` saves typing but is avoided in big projects and header files.

### Q7. What is an inline function?

A suggestion to the compiler to replace a function call with the function's body, removing the call overhead. Good for very small functions. The compiler may ignore it.

## Pointers, references and memory

### Q8. What is a pointer? ★ Most asked

A pointer is a variable that stores the memory address of another variable.

```
int x = 10;
int* p = &x;   // p holds the address of x
cout << *p;    // 10 (dereference)
*p = 20;       // x is now 20
```

### Q9. What is the difference between a pointer and a reference? ★ Most asked

- A reference is an alias (another name) for an existing variable; a pointer stores an address.
- A reference must be initialised when declared and can't be changed to refer to something else; a pointer can be reassigned.
- A reference can't be null; a pointer can be `nullptr`.
- Pointers support arithmetic (`p++`); references don't.

```
int a = 5;
int& r = a;   // r is another name for a
r = 10;       // a is 10
```

### Q10. What is the difference between `new`/`delete` and `malloc`/`free`? ★ Most asked

- `new` is an operator that allocates memory **and calls the constructor**; `malloc` is a C function that only allocates raw memory.
- `new` returns the correct type; `malloc` returns `void*` that needs casting.
- `new` throws `bad_alloc` on failure; `malloc` returns `NULL`.
- `delete` calls the destructor; `free` doesn't.

Never mix them (don't `free` memory from `new`).

### Q11. What is the difference between `malloc` and `calloc`? ★ Most asked

`malloc(size)` allocates one block of memory and leaves it uninitialised (garbage values). `calloc(n, size)` allocates memory for n elements and sets all bytes to **zero**. `calloc` is slightly slower because of the zeroing.

**Interview tip:** "calloc vs malloc" was reported in an embedUR systems interview on Glassdoor.

### Q12. What is a memory leak? How do you avoid it? ★ Most asked

A memory leak is memory allocated with `new` that is never `delete`d, so it's lost until the program ends. Avoid it by pairing every `new` with `delete` (`delete[]` for arrays), using **smart pointers**, and following RAII (resources released in destructors).

### Q13. What are dangling, wild and null pointers?

- **Dangling pointer**: points to memory that has been freed or gone out of scope.
- **Wild pointer**: an uninitialised pointer pointing to a random address.
- **Null pointer**: explicitly points to nothing (`nullptr`).

Always initialise pointers and set them to `nullptr` after deleting.

### Q14. What is the difference between stack and heap memory?

Stack memory stores local variables and function calls; it's fast and freed automatically when the function returns, but limited in size. Heap memory is for dynamic allocation with `new`; it's larger and lasts until you `delete` it, but it's slower and you must manage it.

### Q15. What are smart pointers? ★ Most asked

Classes in `<memory>` that manage dynamic memory automatically and delete it when no longer needed.

- `unique_ptr`: single owner; can't be copied, only moved.
- `shared_ptr`: shared ownership with a reference count; memory is freed when the last owner is gone.
- `weak_ptr`: a non-owning reference to a `shared_ptr` object; breaks circular references.

```
auto p = make_unique<int>(10);
auto s = make_shared<string>("hi");
```

## OOP concepts

### Q16. What are the four pillars of OOP? ★ Most asked

- **Encapsulation**: wrapping data and functions together in a class and controlling access with `private`/`public`. Example: a `BankAccount` with private `balance` and public `deposit()`.
- **Abstraction**: showing only what's needed and hiding the details (abstract classes, interfaces).
- **Inheritance**: one class (child) acquires the properties of another (parent), allowing code reuse.
- **Polymorphism**: one name, many forms (function overloading, operator overloading, virtual functions).

**Interview tip:** Asked in almost every service-company interview (TCS reported). Give a real-life example for each.

### Q17. What is the difference between a class and an object? ★ Most asked

A class is a blueprint or template that defines data members and member functions. An object is an instance of the class that occupies memory. Example: `Car` is a class; `Car myCar;` is an object.

### Q18. What is the difference between a struct and a class in C++? ★ Most asked

They're almost the same. The only difference is the default access: members of a `struct` are **public** by default, while members of a `class` are **private** by default (inheritance defaults follow the same rule). By convention, structs are used for simple data and classes for objects with behaviour.

### Q19. What are access specifiers?

- `public`: accessible from anywhere.
- `private`: accessible only inside the class (and friends).
- `protected`: accessible inside the class and its derived classes.

### Q20. What is a constructor? What are its types? ★ Most asked

A constructor is a special function with the same name as the class and no return type, called automatically when an object is created. Types:

- **Default constructor**: no parameters.
- **Parameterised constructor**: takes arguments.
- **Copy constructor**: creates an object from another object, `ClassName(const ClassName& other)`.

- **Move constructor** (C++11): takes resources from a temporary object, `ClassName(ClassName&& other)`.

### Q21. What is the difference between a shallow copy and a deep copy? ★ Most asked

The default copy constructor makes a **shallow copy**: it copies pointer values, so both objects point to the same memory, which can cause double-delete crashes. A **deep copy** allocates new memory and copies the actual data. Write your own copy constructor and assignment operator when your class owns raw pointers (the Rule of Three).

### Q22. What is a destructor? ★ Most asked

A special function `~ClassName()` that runs automatically when an object is destroyed (goes out of scope or is deleted). It's used to free resources like memory or files. It takes no arguments, has no return type and can't be overloaded.

### Q23. Why should a base class destructor be virtual?

If you delete a derived object through a base class pointer and the base destructor isn't virtual, only the base destructor runs, so the derived part leaks. Making it `virtual` ensures the derived destructor runs first, then the base one.

### Q24. What are the types of inheritance in C++? ★ Most asked

Single, multiple (one class inherits from two or more), multilevel (A → B → C), hierarchical (many children from one parent) and hybrid (a mix).

```
class Animal { public: void eat() {} };
class Dog : public Animal { public: void bark() {} };
```

### Q25. What is the diamond problem? How do you fix it? ★ Most asked

In multiple inheritance, if B and C both inherit from A, and D inherits from both B and C, then D gets two copies of A, making calls ambiguous. Fix it with **virtual inheritance**: `class B : virtual public A` and `class C : virtual public A`, so D has only one copy of A.

## Polymorphism

### Q26. What is the difference between compile-time and runtime polymorphism? ★ Most asked

- **Compile-time (static)**: decided by the compiler; achieved with function overloading, operator overloading and templates.
- **Runtime (dynamic)**: decided while the program runs; achieved with **virtual functions** and function overriding through base class pointers/references.

### Q27. What is the difference between function overloading and overriding? ★ Most asked

- **Overloading**: same function name, different parameters, in the same scope. Compile-time.
- **Overriding**: a derived class redefines a base class's **virtual** function with the same signature. Runtime.

```
class Shape { public: virtual double area() { return 0; } };
class Circle : public Shape {
  double r;
public:
  Circle(double r) : r(r) {}
  double area() override { return 3.14 * r * r; }
};
```

```
Shape* s = new Circle(2);
s->area();   // calls Circle::area at runtime
```

### Q28. What is a virtual function? How does it work internally? ★ Most asked

A virtual function is a member function declared with `virtual` in the base class that can be overridden in derived classes. The call is resolved at runtime based on the actual object type. Internally, each class with virtual functions has a **vtable** (a table of function pointers), and each object has a hidden **vptr** pointing to its class's vtable.

### Q29. What is a pure virtual function and an abstract class? ★ Most asked

A pure virtual function has no body in the base class: `virtual void draw() = 0;`. A class with at least one pure virtual function is an **abstract class**; you can't create its objects, and derived classes must override the function. This is how C++ makes interfaces.

### Q30. What is operator overloading?

Giving a custom meaning to an operator for your own class.

```
class Point {
public:
  int x, y;
  Point operator+(const Point& o) { return {x + o.x, y + o.y}; }
};
```

Operators like `::`, `.`, `.*`, `?:` and `sizeof` can't be overloaded.

### Q31. What is a friend function? ★ Most asked

A function that isn't a member of a class but is allowed to access its private and protected members. It's declared inside the class with the `friend` keyword. Friend classes work the same way. It's useful for operator overloading like `operator<<`, but overusing it breaks encapsulation.

```
class Box {
  int width = 5;
  friend void show(Box b);
};
void show(Box b) { cout << b.width; }
```

**Interview tip:** "What is friend function?" was reported in TCS technical rounds.

### Q32. What is the `this` pointer?

A hidden pointer available inside every non-static member function that points to the object calling the function. It's used to tell apart members from parameters with the same name (`this->x = x;`) and to return the object itself for chaining.

## Templates and STL

### Q33. What are templates? ★ Most asked

Templates let you write generic code that works with any data type. The compiler generates a version for each type used.

```
template <typename T>
T getMax(T a, T b) { return a > b ? a : b; }

getMax(3, 7);       // int
```

```
getMax(2.5, 1.5);   // double
```

There are function templates and class templates (e.g. `vector<T>`).

### Q34. What is the STL? What are its components? ★ Most asked

The Standard Template Library is a collection of ready-made, generic data structures and algorithms. Four components:

- **Containers**: `vector`, `list`, `deque`, `set`, `map`, `unordered_map`, `stack`, `queue`, `priority_queue`.
- **Algorithms**: `sort`, `find`, `binary_search`, `reverse`, `max_element`, `count`.
- **Iterators**: objects to move through containers (`begin()`, `end()`).
- **Functors**: objects that act like functions (used as custom comparators).

### Q35. What is the difference between a vector and an array? ★ Most asked

A C-style array has a fixed size decided at compile time and doesn't know its own size. A `vector` is a dynamic array: it grows automatically (`push_back`), knows its size (`size()`), and manages memory for you. Both give O(1) access by index.

### Q36. How does a vector grow internally?

When a vector is full and you `push_back`, it allocates a bigger block (usually 1.5x or 2x), copies/moves the elements over and frees the old block. So `push_back` is O(1) amortized. `reserve(n)` pre-allocates to avoid repeated resizing.

### Q37. What is the difference between `map` and `unordered_map`? ★ Most asked

`map` is a red-black tree: keys are sorted and operations are O(log n). `unordered_map` is a hash table: no order, O(1) average, O(n) worst case. Use `map` when you need sorted keys or ordered traversal.

### Q38. What is the difference between `set` and `multiset`?

A `set` stores unique sorted elements. A `multiset` allows duplicates. Both are trees with O(log n) operations. `unordered_set` is the hash-based version.

### Q39. How does `priority_queue` work?

It's a max-heap by default: `top()` always gives the largest element. For a min-heap use `priority_queue<int, vector<int>, greater<int>>`. Push and pop are O(log n).

### Q40. How do you sort with a custom comparator?

```
vector<pair<int,int>> v = {{1, 5}, {2, 3}};
sort(v.begin(), v.end(), [](auto& a, auto& b) {
  return a.second < b.second;   // sort by second value
});
```

## Modern C++ and misc

### Q41. What are the main features of C++11 and later? ★ Most asked

`auto` type deduction, range-based for loops, lambda expressions, `nullptr`, smart pointers, move semantics (`&&`, `std::move`), `constexpr`, `unordered_map`, threads (`std::thread`), `override` and `final`. C++17 added structured bindings and `std::optional`; C++20 added concepts, ranges and modules.

### Q42. What is a lambda expression?

An anonymous function written inline: `[capture](params) { body }`.

```
int k = 2;
auto mul = [k](int x) { return x * k; };
mul(5);  // 10
```

`[=]` captures by value, `[&]` by reference.

### Q43. What is move semantics?

Instead of copying a large object's data, a move "steals" the resources (like a pointer to a buffer) from a temporary object and leaves it empty. This makes returning and passing big objects much faster. `std::move(x)` turns `x` into an rvalue so it can be moved.

### Q44. What is RAII?

Resource Acquisition Is Initialization: a resource (memory, file, lock) is acquired in a constructor and released in the destructor. Since destructors run automatically when objects go out of scope, resources are never leaked, even if an exception happens. Smart pointers and `lock_guard` follow RAII.

### Q45. How does exception handling work in C++? ★ Most asked

```
try {
  if (b == 0) throw runtime_error("Divide by zero");
  cout << a / b;
} catch (const runtime_error& e) {
  cout << e.what();
} catch (...) {
  cout << "Unknown error";
}
```

`throw` raises an exception, `try` wraps risky code, and `catch` handles it. Catch by const reference.

### Q46. What is the difference between `++i` and `i++`?

`++i` (pre-increment) increases i and returns the new value. `i++` (post-increment) returns the old value and then increases i. For simple ints there's no real speed difference, but for iterators `++it` avoids making a temporary copy.

### Q47. What is the difference between call by value and call by reference?

Call by value passes a copy, so changes inside the function don't affect the original. Call by reference (`void f(int& x)`) passes the original, so changes do affect it. Pass large objects by `const&` to avoid copies.

### Q48. What is a copy constructor called for?

When an object is initialised from another object (`A b = a;`), passed by value to a function, or returned by value (often skipped by the compiler's copy elision).

## More frequently asked

### Q49. What is a virtual destructor vs a pure virtual destructor?

A virtual destructor makes sure derived destructors run when deleting through a base pointer. A pure virtual destructor (`virtual ~Base() = 0;`) makes the class abstract, but it must still have a body defined outside the class, because derived destructors always call it.

### Q50. What is the difference between `vector::size()` and `capacity()`?

`size()` is the number of elements currently stored. `capacity()` is how many elements fit before the vector needs to reallocate. `shrink_to_fit()` asks to reduce capacity to size.

### Q51. What are iterators and why are they useful?

Iterators are objects that point to elements in a container and can move through them (`begin()`, `end()`, `++it`). They give one common way to traverse any STL container, so the same algorithms (`sort`, `find`) work on different containers.

### Q52. What is the difference between `endl` and `"\n"`?

Both move to a new line, but `endl` also flushes the output buffer, which is slower. In competitive programming and loops, `"\n"` is preferred for speed.